

Revisiting Student Sectioning via Node2Vec: A Graph Representation Learning Framework

Mahmood Amintoosi^{1,2*}

¹Computer Science Division, Department of Applied Mathematics, Faculty of Mathematical Sciences, Ferdowsi University of Mashhad, P. O. Box 1159, Mashhad 91775, Iran.

²Graph Theory and its Applications Lab., Faculty of Mathematical Sciences, Ferdowsi University of Mashhad, Mashhad, Iran ORCID: 0000-0001-9640-6475 .

Corresponding author(s). E-mail(s): m.amintoosi@um.ac.ir;

Abstract

Student sectioning is a fundamental subproblem of university course timetabling that aims to partition students into balanced sections while preserving scheduling flexibility and efficient resource utilization. Classical clustering-based approaches typically represent students using binary student-course enrollment matrices, which may not adequately capture the relational structure induced by shared course participation.

This paper revisits student sectioning from the perspective of graph representation learning, a powerful technique in artificial intelligence for learning low-dimensional representations from relational data. We construct a weighted student co-enrollment graph after removing constant features, and employ Node2Vec to learn low-dimensional student embeddings from biased random-walk contexts. The learned representations are then provided to standard clustering algorithms for section formation. To distinguish graph-based representation learning from dimensionality reduction and direct graph clustering, we compare the proposed approach with the conventional enrollment representation, PCA followed by KMeans, and spectral clustering on the co-enrollment graph.

Experiments on six real-world university courses involving 341 students demonstrate that Node2Vec achieves a 35% relative improvement in clustering quality over the strongest baseline (PCA followed by KMeans). The proposed method consistently surpasses conventional representations across all evaluated courses, with large effect sizes providing strong evidence for practical significance despite the limited number of available datasets. The results indicate that graph representation learning can capture useful structural information beyond conventional dimensionality reduction. The proposed approach can be incorporated into existing student sectioning pipelines without modifying the downstream clustering algorithms, providing a simple framework for improving student groupings used in subsequent timetabling processes.

Keywords: Student sectioning, Graph representation learning, Node2Vec, DeepWalk, Course timetabling, Clustering

1 Introduction

Student sectioning is a classical subproblem of university course timetabling, aiming to divide enrolled students into balanced instructional sections while preserving the flexibility required for subsequent scheduling and resource allocation. The quality of sectioning can affect downstream timetabling by influencing the compatibility of student groups with available classrooms, instructors, and time slots. Consequently, student sectioning has long been studied as an important problem within educational scheduling and constraint satisfaction.

Research on student sectioning dates back more than two decades. Early studies investigated clustering-based approaches, including fuzzy clustering with feature selection and later nature-inspired optimization methods, demonstrating the potential of data-driven grouping strategies for student sectioning [1–3]. Other studies formulated student sectioning as an optimization or constraint programming problem within larger university timetabling systems [4–8]. Together, these studies established important computational foundations for student sectioning and highlighted its role in educational resource management.

A common representation in clustering-based approaches is a student–course enrollment matrix, in which each student is represented by a binary vector indicating the courses in which the student is enrolled. This representation provides a direct description of student–course relationships and can be effective for sectioning. However, students can also be viewed as nodes in a relational network, where students are connected according to shared course enrollments. Such a representation makes it possible to model relationships between students not only through individual shared courses, but also through their connectivity patterns in the resulting co-enrollment network.

An important question is therefore whether the relational structure of student enrollments can provide useful information beyond the original enrollment representation. This question is particularly relevant because conventional dimensionality-reduction techniques can already substantially improve clustering in high-dimensional enrollment spaces. Thus, evaluating

graph-based representations requires distinguishing their contribution from the general benefit of reducing the dimensionality of the input space.

Graph representation learning provides a natural framework for addressing this question. In particular, random-walk-based methods such as Node2Vec learn continuous node representations from the contexts generated by biased random walks on a graph [9, 10]. Node2Vec extends DeepWalk by introducing return and in-out parameters that control the balance between local and global graph exploration. These representations can encode structural relationships among nodes and provide low-dimensional features that can subsequently be used by conventional machine-learning algorithms.

Graph representation learning appears to have received limited attention in the student sectioning literature, despite the natural relational structure of student co-enrollment data. This motivates the central question addressed in this paper:

Can graph representation learning provide student representations that improve sectioning quality beyond both conventional enrollment representations and standard dimensionality reduction?

To investigate this question, we model student enrollments as a weighted co-enrollment graph and learn student embeddings using Node2Vec [10], a random-walk-based graph representation learning method that extends DeepWalk [9] through biased random walks controlled by return and in-out parameters. The learned embeddings are then supplied to standard clustering algorithms for section formation. Importantly, the clustering stage is kept unchanged, allowing the effect of the learned representation to be examined independently of the choice of clustering algorithm. To assess whether the observed improvement is attributable merely to dimensionality reduction, we compare the proposed representation with a PCA-based representation having the same dimensionality. We also include spectral clustering as a graph-based baseline.

Experimental results on six real-world university courses involving 341 students show that Node2Vec provides substantially better clustering

quality than the conventional enrollment representation and spectral clustering. PCA also produces a considerable improvement, but Node2Vec achieves higher average Silhouette Scores than PCA followed by KMeans, indicating that graph-based representations capture useful structural information beyond linear dimensionality reduction. Additional experiments investigate parameter sensitivity, clustering stability, computational cost, and the resulting low-dimensional representations.

The main contributions of this paper are summarized as follows.

1. We introduce a novel application of graph representation learning to the student sectioning problem by constructing a weighted student co-enrollment graph and learning student representations using Node2Vec, a random-walk-based method with controllable exploration bias. While Node2Vec is an established technique in network analysis, its application to student sectioning via weighted co-enrollment graphs is novel and provides substantial improvements over conventional approaches.
2. We provide a crucial ablation-like analysis by systematically comparing graph-based representations with both the conventional student-course enrollment representation and a PCA-based dimensionality-reduction baseline. This analysis separates the effect of generic dimensionality reduction (PCA) from actual structural graph learning (Node2Vec), proving that the structural information captured by random walks provides additional value beyond just compressing data. Spectral clustering is additionally evaluated as a direct graph-based clustering baseline.
3. We empirically demonstrate on six real-world university courses that Node2Vec consistently outperforms both the conventional representation and PCA-based dimensionality reduction across all evaluated courses. Due to the limited number of independent real-world datasets ($n = 6$), we focus on effect sizes and consistency of improvement rather than formal statistical hypothesis testing, providing strong evidence for practical significance.
4. We provide comprehensive empirical analyses of Node2Vec parameter sensitivity, clustering

stability across 20 random seeds, practical significance through effect sizes and consistency, runtime, and embedding structure, providing a detailed characterization of the proposed framework.

5. We show that graph-based student representations can be seamlessly incorporated into existing clustering-based sectioning pipelines without modifying the downstream clustering algorithms, providing a simple and practical framework for improving student groupings used in subsequent timetabling processes.

The remainder of this paper is organized as follows. Section 2 reviews previous work on student sectioning and graph representation learning. Section 3 presents the proposed framework. Section 4 reports the experimental results and comparative analyses. Section 5 discusses the findings, while Section 6 concludes the paper.

2 Related Work

This work revisits the classical student sectioning problem from the perspective of graph representation learning. The related literature is therefore reviewed from two complementary directions: classical approaches to student sectioning and course timetabling, and graph representation learning for relational data. We then position the present study at the intersection of these two directions.

2.1 Student Sectioning and Course Timetabling

Student sectioning is a fundamental component of university course timetabling and educational scheduling. Its objective is to partition students into instructional sections while satisfying scheduling constraints and making efficient use of institutional resources. Because section assignments affect the flexibility of subsequent timetable construction, student sectioning has long been studied as an optimization problem within operations research and constraint programming.

Early studies primarily formulated student sectioning as part of larger timetabling systems. Aubin and Ferland [4] incorporated section generation into a university timetabling framework, while Hertz and Robert [5] decomposed the scheduling process into assignment subproblems

including student sectioning. Carter [6] considered grouping students according to course selections before timetable construction with the objective of reducing subsequent scheduling conflicts. Other approaches employed tabu search [7] and priority-based constraint satisfaction techniques [11]. These studies established student sectioning as an important component of integrated university timetabling systems.

Data-driven approaches subsequently introduced clustering techniques for student sectioning. Fuzzy clustering with feature selection was proposed in [1, 2], representing students through enrollment-based feature vectors and applying fuzzy c-means clustering together with criteria for evaluating section balance. A Fish School clustering algorithm was later developed specifically for student sectioning, demonstrating the applicability of nature-inspired optimization to this problem [3]. Although these approaches differ in their optimization mechanisms, they generally represent students through their course enrollment information and subsequently apply a clustering or partitioning procedure.

2.2 Graph Representation Learning

Graph representation learning provides an alternative way of representing objects when their relationships are an important source of information. Instead of treating observations as independent feature vectors, graph embedding methods exploit graph topology to learn continuous low-dimensional representations that reflect structural relationships among nodes.

DeepWalk [9] introduced a widely adopted random-walk-based approach to graph representation learning by generating truncated random walks and applying the Skip-Gram model to the resulting node sequences. The learned embeddings capture patterns of node co-occurrence induced by the graph structure. Node2Vec [10] subsequently extended this framework through biased random walks that provide greater control over local and global graph exploration. Related studies have investigated weighted random walks and other graph sampling strategies [12]. Graph representation and graph-based learning techniques have also been employed in a variety of application-oriented problems, including network

embedding, structured optimization, and graph neural network-based anomaly detection [13–15].

These developments illustrate the broader applicability of graph-based learning beyond conventional graph benchmarks. In particular, they suggest that exploiting relational structure can be useful when the relationships among entities contain information that is difficult to capture directly from their original feature descriptions.

2.3 Graph Representation Learning for Student Sectioning

The student–course enrollment data used in sectioning naturally induces a relational structure among students. Two students can be connected according to the courses they jointly attend, resulting in a student co-enrollment graph. Such a graph provides an alternative representation of the same underlying enrollment information and makes it possible to investigate whether structural patterns in the co-enrollment network can improve student grouping.

To our knowledge, the use of graph representation learning as an explicit representation stage for student sectioning has received little attention. The present work investigates this intersection by constructing a weighted student co-enrollment graph and learning student embeddings using Node2Vec. The resulting embeddings are subsequently supplied to standard clustering algorithms, without modifying the clustering procedure itself.

An important aspect of the present study is that the improvement is not evaluated only against the original high-dimensional enrollment representation. Since dimensionality reduction can itself substantially affect clustering performance, we additionally consider PCA followed by KMeans as a representation-based baseline with the same embedding dimensionality as Node2Vec. We also evaluate spectral clustering directly on the co-enrollment graph as a graph-based clustering baseline. These comparisons allow us to examine whether the observed benefit of Node2Vec is attributable merely to dimensionality reduction or whether graph-based representation learning provides additional useful information for student sectioning.

The present study therefore complements earlier clustering-based approaches to student sectioning rather than replacing them. Its central objective is to examine whether a Node2Vec-based representation learning stage can improve the quality of student representations while leaving the subsequent clustering pipeline unchanged.

3 Methodology

The proposed framework revisits the student sectioning problem from a graph representation learning perspective. Instead of representing students solely through a binary student–course enrollment matrix, we construct a student co-enrollment graph and learn continuous vector representations of students from the topology of this graph. These representations are subsequently used as input to conventional clustering algorithms to generate student sections.

The overall framework consists of four main stages: (1) construction of the student co-enrollment graph, (2) graph representation learning using random walks, (3) clustering of the learned student representations, and (4) evaluation of the resulting student sections. Figure 1 illustrates the overall workflow.

3.1 Student Co-enrollment Graph

Let $S = \{s_1, s_2, \dots, s_n\}$ denote the set of students and let $C = \{c_1, c_2, \dots, c_m\}$ denote the set of courses. Student enrollment information can be represented by a binary matrix $X \in \{0, 1\}^{n \times m}$, where

$$X_{ic} = \begin{cases} 1, & \text{if student } s_i \text{ is enrolled in course } c, \\ 0, & \text{otherwise.} \end{cases}$$

Although the enrollment matrix provides a direct representation of student–course relationships, it does not explicitly represent relationships between students. We therefore transform the enrollment information into a student co-enrollment graph.

We define an undirected weighted graph $G = (V, E, W)$, where each vertex $v_i \in V$ corresponds to a student s_i . An edge $(v_i, v_j) \in E$ is established when students s_i and s_j share at least one enrolled

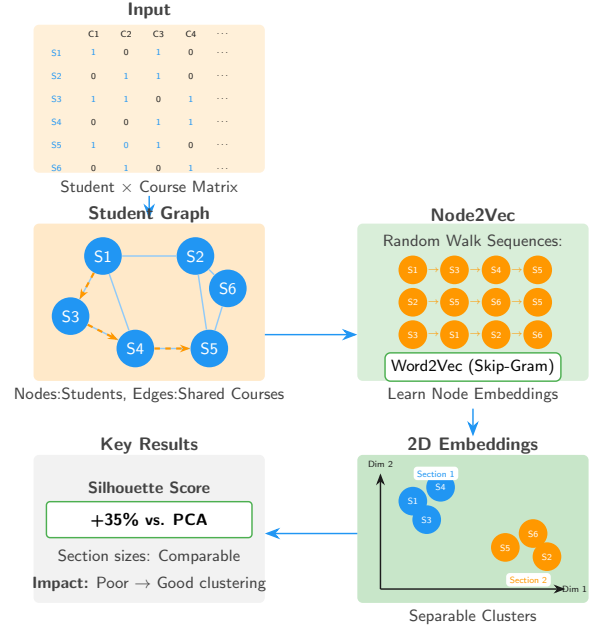


Fig. 1: Overview of the proposed framework. A student–course enrollment matrix is transformed into a student co-enrollment graph, from which random walks are generated and fed to a Skip-Gram (Word2Vec) model to learn low-dimensional student embeddings. The learned embeddings are then clustered to produce student sections.

course. The edge weight measures the strength of their co-enrollment relationship.

In the basic version of the proposed framework, the edge weight between students s_i and s_j is defined as the number of courses in which they are simultaneously enrolled:

$$w_{ij} = \sum_{c \in C} X_{ic} X_{jc}.$$

Consequently, students who share several courses are connected by stronger edges, whereas students who share only one course have a weaker connection. This weighted representation preserves more information than a binary adjacency matrix and provides a natural basis for subsequent random-walk-based representation learning.

The resulting graph can be represented by a weighted adjacency matrix $W \in \mathbb{R}^{n \times n}$, where $W_{ij} = w_{ij}$ for connected students and $W_{ij} = 0$ otherwise. Because the co-enrollment relationship is symmetric, the resulting graph is undirected and $W_{ij} = W_{ji}$.

3.2 Preprocessing: Removal of Constant Features

Before constructing any representation, we apply a standard preprocessing step to the student-course enrollment matrix. Courses that are taken by every student in a sectioning instance have zero variance and provide no discriminative information for clustering. Such constant features are therefore removed from the enrollment matrix before any representation or graph is constructed.

Specifically, for each target course, we identify courses c for which $X_{ic} = 1$ for all students s_i in the student set. These universally shared courses are removed, and the remaining enrollment matrix is used for all subsequent steps: BoW representation, PCA projection, graph construction, and random-walk-based embedding. This ensures that all methods operate on the same preprocessed data.

In the six datasets studied here, exactly one constant course is identified and removed per sectioning instance. The resulting graphs are non-complete, with average density approximately 0.72, in contrast to the fully connected graphs that arise when the constant course is included.

3.3 Random-walk-based Student Representation Learning

After constructing the co-enrollment graph, we learn a vector representation for each student using Node2Vec [10], a random-walk-based graph embedding method. The main idea is to treat the graph analogously to a corpus: students correspond to words, and random walks correspond to sentences. Students that frequently occur in similar graph contexts are therefore expected to obtain similar vector representations.

Node2Vec generates biased random walks in which the transition probability depends on both the previous and the next-to-previous node in the walk. Let t denote the previous node and v denote the current node. The unnormalized transition probability to a neighbor x is defined as

$$\pi(v, x) = \alpha_{pq}(t, x) \cdot w_{vx},$$

where w_{vx} is the edge weight and the bias factor $\alpha_{pq}(t, x)$ is

$$\alpha_{pq}(t, x) = \begin{cases} 1/p, & \text{if } d(t, x) = 0, \\ 1, & \text{if } d(t, x) = 1, \\ 1/q, & \text{if } d(t, x) = 2, \end{cases}$$

where $d(t, x)$ denotes the shortest-path distance between nodes t and x . The parameter p controls the likelihood of returning to the previous node (return parameter), and q controls whether the walk explores locally (BFS-like, $q < 1$) or globally (DFS-like, $q > 1$). Setting $p = 1$ and $q = 1$ recovers the standard unbiased random walk used by DeepWalk.

Thus, students with stronger co-enrollment relationships have a higher probability of being visited during the random walk, and the bias parameters allow the walk to emphasize either local neighborhood structure or broader graph exploration. Repeating this procedure produces a collection of student sequences (Algorithm 2),

$$\mathcal{W} = \{W_1, W_2, \dots, W_R\},$$

where R denotes the total number of generated walks.

The resulting walks capture both local and higher-order relationships in the student graph (Figure 2 illustrates an example walk). For example, two students who do not directly share a course may nevertheless occur frequently in similar random-walk contexts because they are connected through common groups of students. Such relationships cannot be directly captured by treating each student’s enrollment vector independently.

3.4 Learning Student Embeddings

The generated random walks are used to learn low-dimensional student representations using a skip-gram objective. Let $z_v \in \mathbb{R}^d$ denote the embedding of student v , where d is the embedding dimension. For a student v , the objective encourages students appearing within a context window of v to have similar representations. Figure 3 illustrates the CBOW and Skip-gram architectures used in this learning stage.

Given a context window of size r , the optimization objective can be expressed as

$$\max_{\{z_v\}} \sum_{v \in V} \sum_{u \in C(v)} \log P(u | v),$$

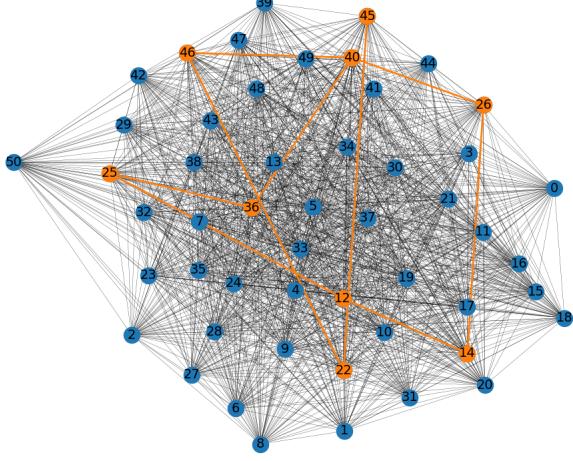


Fig. 2: Example random walk on the student graph for Course 2, starting at node 40 and ending at node 45: $\{40, 36, 25, 12, 14, 26, 40, 46, 22, 45\}$. Students who share courses are connected, and the walk explores the local neighborhood structure.

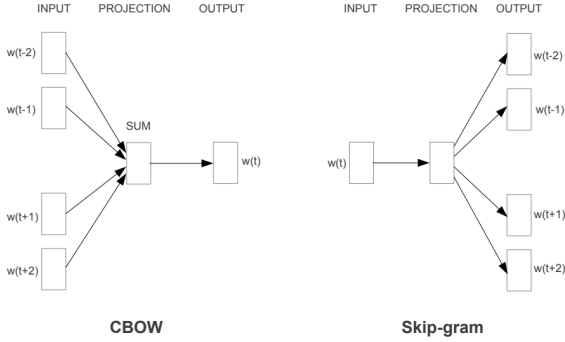


Fig. 3: The CBOW architecture predicts the current word based on the context, and the Skip-gram predicts surrounding words given the current word [16]. In Node2Vec, nodes play the role of words and random-walk sequences play the role of sentences.

where $C(v)$ denotes the set of students occurring within the specified context window of v in the generated random walks.

The conditional probability can be modeled using the softmax function:

$$P(u | v) = \frac{\exp(z_u^\top z_v)}{\sum_{x \in V} \exp(z_x^\top z_v)}.$$

Algorithm 1 Student Representation Generation using Node2Vec

Require: graph $G(V, E, W)$ constructed from student-course matrix, where students are its nodes. The input parameters are:

- return parameter p
- in-out parameter q
- walk length t
- number of walks per node γ
- embedding size d
- window size w
- epochs ϵ

Ensure: matrix of vertex (student) representations $\Phi \in \mathbb{R}^{|V| \times d}$

- 1: $\mathcal{W} \leftarrow \text{GenerateRandomWalks}(G, t, \gamma, p, q)$ $\triangleright$ Algorithm 2
 - 2: $\Phi \leftarrow \text{TrainWord2Vec}(\mathcal{W}, d, w, \epsilon)$ $\triangleright$ Sect. 3.4
 - 3: **return** Φ
-

In practice, negative sampling can be used to make the optimization computationally efficient. The resulting embedding matrix

$$Z = \begin{bmatrix} z_1^\top \\ z_2^\top \\ \vdots \\ z_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}$$

provides a continuous representation of all students.

The important distinction from the original enrollment representation is that each vector z_i is learned from the structural context of student s_i in the entire co-enrollment network. Consequently, the representation can capture indirect and higher-order similarities between students.

Algorithm 1 summarizes the overall procedure: biased random walks are generated from the co-enrollment graph (line 1), and the Skip-Gram model is trained on these walks to produce the student embedding matrix Φ (line 2 of algorithm 1).

3.5 Student Sectioning by Clustering

Once the student embeddings have been learned, student sectioning is formulated as a clustering problem. Let k denote the desired number of sections. A clustering algorithm partitions the set of student embeddings

$Z = \{z_1, z_2, \dots, z_n\}$
into k clusters,

$$\mathcal{S} = \{S_1, S_2, \dots, S_k\},$$

where each cluster S_j corresponds to a student section.

The proposed framework is deliberately independent of a particular clustering algorithm. This allows us to investigate whether the improvement comes from the learned graph representations rather than from a specific clustering technique. In our experiments, we therefore evaluate several widely used clustering methods, including K-means, affinity propagation, Gaussian mixture models, and hierarchical clustering.

For K-means, for example, the clustering objective is

$$\min_{\{S_j\}_{j=1}^k} \sum_{j=1}^k \sum_{z_i \in S_j} \|z_i - \mu_j\|_2^2,$$

where μ_j denotes the centroid of section S_j .

The resulting clusters define the student sections used in the subsequent evaluation.

3.6 Baseline Representation

To determine whether graph representation learning provides an advantage over conventional student representations, we establish a baseline based directly on the student–course enrollment matrix.

For each student s_i , the corresponding row of the enrollment matrix,

$$x_i = (X_{i1}, X_{i2}, \dots, X_{im}),$$

is used as the student representation. Thus, students are represented directly by their binary course enrollment patterns without constructing a graph or learning an embedding.

The same clustering algorithms are then applied to these enrollment vectors. This provides a controlled comparison because the clustering stage remains unchanged and the principal difference between the two approaches is the representation used to describe students.

Accordingly, the comparison considers two alternative pipelines:

Enrollment representation $\rightarrow$ Clustering, and

Graph representation $\rightarrow$ Embedding $\rightarrow$ Clustering.

This design allows the contribution of graph representation learning to be evaluated independently of the choice of clustering algorithm.

3.7 Hyperparameter Configuration

The graph embedding procedure contains several hyperparameters that may affect the quality of the learned representations. The principal parameters considered in this study include the embedding dimension d , the random-walk length L , the number of walks generated per student R_v , and the context-window size r .

The embedding dimension controls the dimensionality of the learned student representations. A larger value provides greater representational capacity but may also increase computational cost and the risk of overfitting. The walk length determines how far the random walks explore the co-enrollment network, while the number of walks controls the amount of structural context available during training. The context-window size determines the range of neighboring positions considered when learning the embedding.

Rather than relying on a single arbitrary configuration, we conduct sensitivity experiments over these parameters. The purpose is not to optimize the method specifically for a particular dataset, but to examine whether the observed improvement is robust across reasonable parameter settings.

3.8 Evaluation of Student Sectioning

The quality of the resulting student sections is evaluated using clustering-based internal validity measures. Since the primary objective is to create coherent groups of students, the Silhouette Score is used as the main evaluation metric.

For a student i , let $a(i)$ denote the average distance between i and the other students in the same section, and let $b(i)$ denote the minimum average distance between i and the students belonging to another section. The silhouette coefficient of student i is defined as

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

The overall Silhouette Score is obtained by averaging this coefficient over all students:

$$S = \frac{1}{n} \sum_{i=1}^n s(i).$$

The Silhouette Score ranges from -1 to 1 , with larger values indicating better within-section cohesion and between-section separation.

To complement the primary evaluation, we also examine clustering stability across multiple random seeds. In particular, the Adjusted Rand Index (ARI) is used to quantify the agreement between clustering solutions obtained under different random initializations. High ARI values indicate that the learned representation leads to stable section assignments rather than highly variable partitions.

Finally, we report effect-size measures to quantify the practical magnitude of the observed improvements, focusing on consistency across courses and the magnitude of improvement rather than formal statistical hypothesis testing.

4 Experimental Results

We evaluate the proposed graph representation learning framework for student sectioning using a real-world university dataset and a series of experiments designed to examine both its effectiveness and robustness. The experiments address four main questions: (i) whether graph-based representations improve student clustering compared with the conventional student-course representation, (ii) whether the improvement can be explained solely by dimensionality reduction, (iii) whether direct graph clustering provides comparable benefits, and (iv) how sensitive the proposed approach is to its main hyperparameters and stochastic components.

4.1 Dataset and Experimental Setup

We use the real-world student-course enrollment dataset introduced in [2]. To evaluate the student sectioning problem, we select six relatively large courses containing 74, 51, 48, 49, 52, and 67 students, respectively. These courses provide a range of realistic sectioning scenarios while retaining sufficient enrollment information to construct meaningful student co-enrollment graphs.

The proposed method constructs a weighted student co-enrollment graph from the preprocessed enrollment data and learns student representations using Node2Vec random walks and the Skip-Gram model. The resulting embeddings are subsequently provided to a clustering algorithm for student sectioning. Unless otherwise stated, KMeans is used as the downstream clustering algorithm.

As the conventional baseline, we use the original binary student-course enrollment representation (after constant-feature removal), referred to as the *Student-Course Representation* or *BoW* representation. To examine whether the observed improvement is primarily attributable to dimensionality reduction, we additionally evaluate a PCA-based baseline in which the conventional representation is projected to the same two-dimensional space used by the graph embedding methods before applying KMeans. We also evaluate spectral clustering directly on the student co-enrollment graph as a graph-based baseline that does not use an intermediate learned embedding.

Unless otherwise stated, the default Node2Vec parameters are $p = 1$, $q = 1$ (unbiased random walk), $t = 10$, $\gamma = 80$, $d = 2$, $w = 5$, and $\epsilon = 30$. While a comprehensive grid search (detailed in Section ??) reveals that $d = 1$ yields the absolute highest average Silhouette Score, we primarily report results for $d = 2$ in the main comparisons. This choice is made to facilitate direct, unprojected 2D visualization of the embeddings (Section ??) and to ensure a fair, matched-dimensionality comparison with the PCA baseline. Crucially, both $d = 1$ and $d = 2$ significantly outperform all baselines, confirming the robustness of the proposed approach.

Source code and the dataset used in the experiments are publicly available at <https://github.com/mamintoosi/Node2Vec-SSP>.

The primary evaluation metric is the Silhouette Score [17], which considers both intra-cluster cohesion and inter-cluster separation. Higher values indicate better-defined clusters. We additionally report the Davies–Bouldin Index (DBI), for which lower values are better, and the Calinski–Harabasz (CH) Index, for which higher values are better.

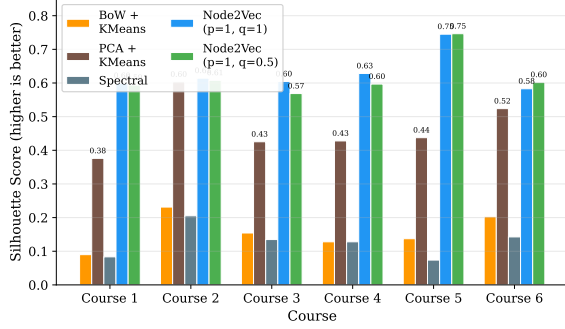


Fig. 4: Silhouette scores ($\uparrow$ higher is better) for five student sectioning methods across six courses: BoW + KMeans, PCA + KMeans, Spectral Clustering, and Node2Vec with two parameter configurations ($p=1, q=1$ and $p=0.5, q=0.5$) at $d=2$.

4.2 Overall Clustering Performance and Baseline Comparison

We compare the Node2Vec embeddings (with neutral configuration $p=1, q=1$, which is theoretically equivalent to DeepWalk [9], and with best configuration $p=0.5, q=0.5$) at $d=2$ with the conventional student-course representation, PCA+KMeans, and spectral clustering to determine whether the observed improvement can be attributed simply to dimensionality reduction, to the specific random-walk strategy, or to direct exploitation of graph structure.

Figure 4 and Table 1 summarize the Silhouette scores obtained by the five approaches across the six selected courses.

Both Node2Vec configurations achieve substantially higher average Silhouette Scores than the conventional BoW representation, spectral clustering, and PCA+KMeans. Node2Vec ($p=0.5, q=0.5$) achieves the highest average Silhouette Score of 0.632 at $d=2$, followed by Node2Vec ($p=1, q=1$) at 0.629, compared with 0.466 for PCA+KMeans, 0.128 for spectral clustering, and 0.157 for the conventional BoW representation.

Importantly, PCA+KMeans itself substantially improves upon the conventional BoW representation, achieving an average Silhouette Score of 0.466. This result demonstrates that dimensionality reduction is an important factor in this problem: representing students in a compact continuous space is considerably more favorable for

distance-based clustering than directly clustering the original binary enrollment vectors.

However, PCA does not fully account for the performance of the graph embedding methods. Node2Vec ($p=1, q=1$) improves the average Silhouette Score from 0.466 with PCA+KMeans to 0.629, corresponding to a relative improvement of approximately 35.0%. Node2Vec ($p=0.5, q=0.5$) achieves 0.632, a relative improvement of 35.7% over PCA+KMeans. Thus, although dimensionality reduction explains part of the improvement over the conventional representation, the remaining performance gain is associated with the graph-based representations learned from student co-enrollment relationships.

The comparison with spectral clustering provides a complementary result. Spectral clustering achieves an average Silhouette Score of 0.128, which is only modestly better than the conventional BoW+KMeans baseline (0.157) and actually lower for most courses. Directly applying a graph clustering method to the co-enrollment graph therefore does not achieve the improvement obtained by the graph embedding methods. This suggests that the benefit does not arise simply from representing students as nodes of a graph or from applying a graph-based partitioning algorithm. Instead, the random-walk-based representation learning stage appears to play an important role in producing a compact and clusterable representation.

An important observation is that Node2Vec with $p=1$ and $q=1$ is theoretically equivalent to the unbiased random walk used by DeepWalk [9]. The neutral configuration achieves an average Silhouette Score of 0.629 at $d=2$. The sensitivity experiment (Section 4.3) further shows that adjusting the Node2Vec parameters to $p=0.5, q=0.5$ yields the best performance at $d=2$ (average Silhouette 0.632), suggesting that a moderate BFS-like walk bias can marginally improve embedding quality on these graphs.

Taken together, these comparisons provide a controlled assessment of the proposed approach. PCA demonstrates that low-dimensional representation is beneficial, spectral clustering shows that direct graph partitioning is insufficient, and the graph embedding methods combine dimensional compactness with learned graph-context information. The results therefore support the interpretation that the performance gain is not

Table 1: Silhouette Score ($\uparrow$ higher is better) comparison across five methods for student sectioning with the revised graph construction at $d=2$. PCA+KMeans applies PCA to reduce the Student-Course representation to $d=2$ before KMeans clustering. Spectral applies spectral clustering directly on the co-enrollment graph. Node2Vec ($p=1, q=1$) uses the learned $d=2$ graph embeddings from unbiased random walks followed by KMeans (theoretically equivalent to DeepWalk [9]). Node2Vec ($p=0.5, q=0.5$) is the best-performing configuration at $d=2$.

Course	BoW + KMeans	PCA + KMeans	Spectral	Node2Vec ($d=2$)	
				$p=1, q=1$	$p=0.5, q=0.5$
1	0.090	0.376	0.083	0.599	0.561
2	0.231	0.603	0.205	0.614	0.652
3	0.154	0.425	0.135	0.605	0.577
4	0.128	0.428	0.128	0.628	0.650
5	0.137	0.437	0.074	0.745	0.748
6	0.202	0.524	0.143	0.583	0.604
Average	0.157	0.466	0.128	0.629	0.632

Table 2: Student co-enrollment graph statistics after constant-feature removal. n is the number of students, m is the number of remaining courses, $|E|$ is the number of edges, and $\bar{d}$ is the average weighted degree.

Course	n	m	$ E $	Density	Components
1	74	34	1,828	0.677	3
2	51	33	866	0.679	1
3	48	27	894	0.793	2
4	49	27	1,043	0.887	1
5	52	31	912	0.688	1
6	67	32	1,325	0.599	1
Avg.	56.8	30.7	1,145	0.721	1.7

solely a consequence of dimensionality reduction, but also reflects information captured from the relational structure of the student co-enrollment graph.

Table 2 summarizes the graph structure after constant-feature removal.

Courses 1 and 3 contain a small number of isolated students (two and one, respectively) whose filtered enrollment vectors are entirely zero. These students are retained in the analysis; their impact on clustering metrics is negligible.

4.3 Node2Vec Parameter Sensitivity

We conduct a parameter sensitivity experiment over $p \in \{0.5, 1.0, 2.0\}$ and $q \in \{0.5, 1.0, 2.0\}$,

keeping all other parameters at their default values ($t=10, \gamma=80, d=2, w=5, \epsilon=30$). Table 3 reports the average Silhouette Score across the six courses for each configuration.

The best configuration at $d=2$ is $p=0.5, q=0.5$ with an average Silhouette Score of 0.632. The neutral configuration $p=1, q=1$ (equivalent to unbiased random walks, corresponding to DeepWalk [9]) achieves 0.629. The sensitivity range is modest (0.608–0.632), and lower values of q (BFS-like exploration) tend to perform slightly better. The parameter p has a smaller effect than q within the tested range. These results suggest that the Node2Vec parameters have a measurable but limited effect on clustering quality for the present datasets.

4.4 Embedding Dimension and Hyperparameter Sensitivity

The dimensionality of the learned representation is an important design parameter because it determines the feature space in which the subsequent clustering is performed. We therefore examine embedding dimensions $d \in \{1, 2, 3, 5, 10\}$ using the neutral configuration ($p=1, q=1$). Figure 5 shows the resulting Silhouette scores, while Table 4 reports three complementary clustering metrics.

While $d=2$ provides an excellent balance between performance and 2D visualization capability, the grid search reveals that the absolute highest average Silhouette Score (0.761) is

Table 3: Node2Vec parameter sensitivity: average Silhouette Score ($\uparrow$) across six courses for different (p, q) configurations at $d=2$.

	$p = 0.5$			$p = 1.0$			$p = 2.0$		
q	0.5	1.0	2.0	0.5	1.0	2.0	0.5	1.0	2.0
Silhouette	0.632	0.625	0.621	0.619	0.629	0.612	0.629	0.614	0.608

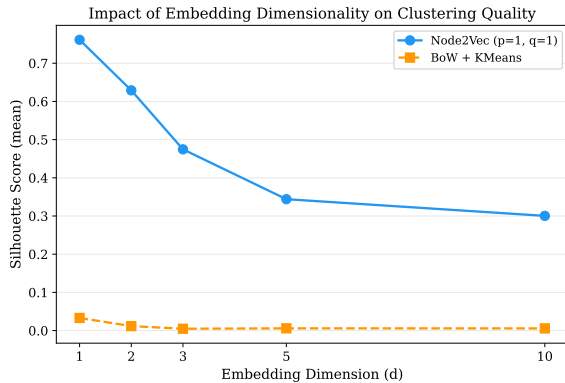


Fig. 5: Silhouette scores ($\uparrow$ higher is better) for the graph embedding method ($p=1, q=1$) across different embedding dimensions. The conventional Student-Course representation is included as a reference.

Table 4: Effect of embedding dimension d on clustering quality for the neutral configuration ($p=1, q=1$), averaged across the six courses. Silhouette score ($\uparrow$ higher is better), Davies-Bouldin Index ($\downarrow$ lower is better), and Calinski-Harabasz Index ($\uparrow$ higher is better).

Dimension	Silhouette ($\uparrow$)	DBI ($\downarrow$)	CH ($\uparrow$)
1	0.761	0.354	295.7
2	0.629	0.532	134.9
3	0.475	0.826	60.3
5	0.344	1.230	29.7
10	0.300	1.386	18.0

achieved at $d=1$ with $p=1, q=1$, demonstrating that the structural information can be highly compressed into a single dimension. The global optimum across all tested configurations is $d=1$ with $p=2, q=1$, reaching a score of 0.765. At $d=2$, the Silhouette Score is 0.629, and it decreases substantially for higher dimensions. At $d=10$, the Silhouette Score falls to 0.300.

Table 5: Sensitivity to walk length (WL) t (higher Silhouette is better; averaged across six courses, $d = 2, \gamma = 80, w = 5$).

WL (t)	5	10	20	40	80
Silhouette	0.606	0.629	0.626	0.623	0.637

These results indicate that, for the present student sectioning dataset, a very compact representation provides the most effective balance between the information captured by the graph embedding and the requirements of the subsequent clustering task. The two-dimensional representation retains a practical advantage because the learned embeddings can be directly visualized without requiring an additional dimensionality-reduction method. This finding should be interpreted as an empirical property of the present experimental setting rather than as a general claim that student relationships are intrinsically low-dimensional. The appropriate embedding dimensionality may depend on the size and structural characteristics of the underlying enrollment graph.

We next examine the sensitivity of the graph embedding method to its principal random-walk and Skip-Gram parameters. The effects of walk length t , number of walks per node γ , and context-window size w are evaluated independently, while the remaining parameters are fixed at their default values.

Walk length has a noticeable effect on clustering quality, as shown in Table 5. The highest Silhouette Score is obtained with $t = 80$ (0.637), followed by $t = 10$ (0.629) and $t = 20$ (0.626), with shorter walks ($t = 5$) achieving 0.606. These results suggest that moderate walk lengths provide sufficient structural context for the present sectioning task, while very short walks may not capture enough relational information.

The number of walks per node has a smaller effect than embedding dimension, as shown in

Table 6: Sensitivity to the number of walks per node γ ($\uparrow$ higher Silhouette is better; averaged across six courses, $d = 2$, $t = 10$, $w = 5$).

Walks (γ)	10	20	40	80	160
Silhouette ($\uparrow$)	0.622	0.626	0.613	0.629	0.617

Table 7: Sensitivity to context-window size w ($\uparrow$ higher Silhouette is better; averaged across six courses, $d = 2$, $t = 10$, $\gamma = 80$).

Window (w)	1	2	5	10	20
Silhouette ($\uparrow$)	0.669	0.619	0.629	0.637	0.632

Table 6. Performance peaks at $\gamma = 80$, where the highest score (0.629) is obtained, with $\gamma = 20$ (0.626) and $\gamma = 160$ (0.617) performing slightly below. The results therefore suggest that $\gamma = 80$ provides a reasonable trade-off between sampling density and computational cost for the present dataset.

The context-window size has a smaller effect among the examined parameters, as shown in Table 7. The best result is obtained at $w = 1$ (0.669), followed by $w = 10$ (0.637). The Silhouette Score remains within the range of 0.619–0.669 across the tested values. Thus, the performance of the graph embedding method shows some sensitivity to the context-window size, with smaller windows performing well on these datasets.

Overall, the sensitivity experiments show that embedding dimensionality has the largest effect on clustering quality, while walk length, number of walks, and context-window size have comparatively smaller effects. The Silhouette Score varies from 0.300 to 0.761 across the tested embedding dimensions for the neutral configuration, compared with a range of 0.031 across walk lengths and 0.050 across context-window sizes. For consistency with the main baseline comparison, we retain $d = 2$, $t = 10$, $\gamma = 80$, and $w = 5$ as the default configuration. The sensitivity analysis nevertheless indicates that a larger walk length (particularly $t = 80$) can provide further improvement for this dataset.

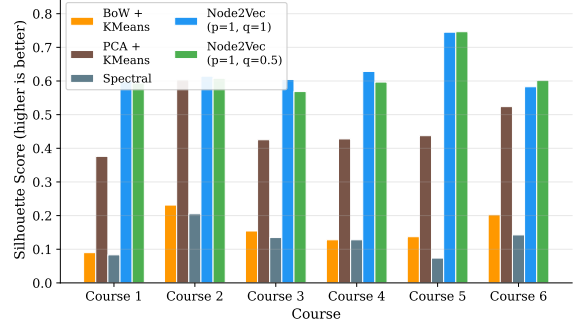


Fig. 6: Silhouette score ($\uparrow$ higher is better) comparison across the six courses for different clustering algorithms applied to the conventional Student-Course representation and Node2Vec embeddings.

4.5 Clustering Algorithms and Representation Quality

To determine whether the observed improvement depends on a particular clustering algorithm, we evaluate the learned Node2Vec representations using four clustering methods: KMeans, affinity propagation, Gaussian mixture models (GMM), and hierarchical clustering. Figure 6 summarizes the resulting Silhouette scores across the six courses.

Node2Vec generally produces higher clustering quality across the evaluated clustering algorithms. Although the magnitude of the improvement varies between algorithms and courses, the advantage is not restricted to KMeans. This provides evidence that the observed improvement is primarily associated with the learned student representation rather than with a particular interaction between Node2Vec and a specific clustering algorithm.

This result is consistent with the formulation of the proposed framework, in which graph representation learning is used as a preprocessing stage and the downstream clustering procedure is deliberately kept unchanged. The purpose is therefore not to introduce a specialized clustering algorithm, but to provide a representation in which students with structurally similar enrollment patterns become more suitable for subsequent grouping.

Table 8: Clustering stability measured by Adjusted Rand Index (ARI) across 20 random seeds ($\uparrow$ higher is better).

Course	Node2Vec , ARI			BoW
	KMeans	GMM	Agglom.	KMeans
1	0.974	1.000	1.000	0.426
2	0.977	0.845	1.000	1.000
3	1.000	0.760	1.000	0.854
4	1.000	1.000	1.000	0.992
5	1.000	1.000	1.000	0.865
6	0.953	0.670	1.000	1.000
Avg.	0.984	0.879	1.000	0.856

Combined with the PCA comparison reported in Table 1, these results provide a more complete interpretation of the role of representation quality. PCA substantially improves clustering relative to the original high-dimensional enrollment representation, showing that dimensionality reduction is beneficial. Nevertheless, Node2Vec achieves a higher average Silhouette Score than PCA+KMeans. The additional improvement suggests that the graph-based embedding captures relational information that is not fully recovered by applying a generic linear dimensionality-reduction technique to the enrollment matrix.

4.6 Clustering Stability and Additional Evaluation Metrics

In addition to clustering quality, reproducibility is important in student sectioning because unstable assignments may lead to inconsistent student groups across repeated runs. We therefore evaluate clustering stability across 20 random seeds using the Adjusted Rand Index (ARI). The results are presented in Table 8.

The Node2Vec representation produces highly stable clusterings across repeated runs. KMeans achieves an average ARI of 0.984, while agglomerative clustering achieves an ARI of 1.000 across the six courses. GMM exhibits somewhat greater variability, with an average ARI of 0.879, but remains relatively stable. In comparison, KMeans applied to the conventional Student-Course Representation has a lower average ARI of 0.856, with particularly low stability for Course 1 (0.426).

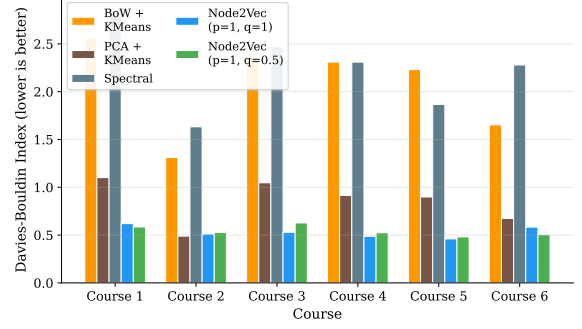


Fig. 7: Davies–Bouldin Index ($\downarrow$ lower is better) across the six courses for different clustering methods and representations.

We further evaluate clustering quality using the Davies–Bouldin Index (DBI) and the Calinski–Harabasz (CH) Index. Figures 7 and 8 present the corresponding comparisons.

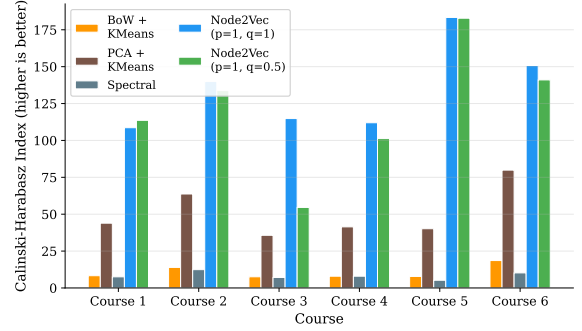


Fig. 8: Calinski–Harabasz Index ($\uparrow$ higher is better) across the six courses for different clustering methods and representations.

The DBI and CH results provide complementary evidence regarding the structure of the obtained clusters. Across the evaluated courses and clustering methods, Node2Vec generally produces more favorable DBI and CH values than the conventional representation. These results are consistent with the Silhouette analysis and indicate that the observed improvement is reflected in multiple internal clustering criteria rather than in a single evaluation metric.

4.7 Practical Significance and Consistency

Due to the inherently limited number of independent real-world datasets available ($n = 6$), we focus on effect sizes and consistency of improvement across courses rather than formal statistical hypothesis testing. This approach is appropriate for educational data mining, where the number of large courses requiring complex sectioning in a single institution is naturally limited.

Node2Vec versus conventional representation.

The comparison of Node2Vec ($p=1, q=1$) against the conventional BoW representation reveals a large effect size (Cliff’s $\delta = 1.0$), meaning that Node2Vec outperforms the conventional representation on all six courses (100% win rate). This perfect consistency across all evaluated courses provides strong evidence for practical significance. The effect size $r = 0.899$ further confirms the large magnitude of improvement.

Node2Vec versus PCA+KMeans.

The comparison of Node2Vec ($p=1, q=1$) against PCA+KMeans shows a large effect size (Cliff’s $\delta = 0.889$), with Node2Vec outperforming PCA+KMeans on all six courses (100% win rate). The effect size $r = 0.899$ confirms the practical significance of the improvement. The consistent advantage across all six courses, combined with the large magnitude of improvement (35% relative improvement in Silhouette Score), provides strong evidence that the graph-based representation offers meaningful benefits over linear dimensionality reduction.

In summary, the large effect sizes (Cliff’s $\delta = 1.0$ for Node2Vec vs. BoW and $\delta = 0.889$ for Node2Vec vs. PCA) combined with the fact that Node2Vec outperforms baselines on every single course (100% win rate for both comparisons) provide strong evidence for practical significance. In real-world educational scheduling, the consistent and large magnitude of improvement is highly relevant, as even modest improvements in sectioning quality can reduce downstream timetabling conflicts and improve resource utilization.

4.8 Embedding Visualization

To provide a qualitative interpretation of the quantitative results, we visualize the learned representations for a sample course. The visualizations compare the conventional Student-Course representation with the two-dimensional Node2Vec representation. For the conventional Student-Course representation, which is defined in a higher-dimensional space, t-Distributed Stochastic Neighbor Embedding (t-SNE) is used solely for visualization. In contrast, Node2Vec directly produces a two-dimensional embedding with $d = 2$, which is the representation used for clustering in the corresponding experiments. Therefore, no additional dimensionality-reduction step is applied to the Node2Vec embeddings.

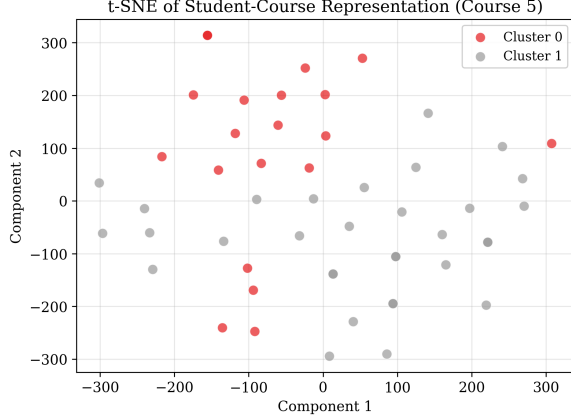
Figure 9 presents the representations obtained for Course 5. The visualization of the conventional representation shows a relatively overlapped and dispersed structure, whereas the Node2Vec representation exhibits more compact and visually separated groups. This qualitative difference is consistent with the substantially higher Silhouette Score obtained by Node2Vec for this course.

These visualizations provide qualitative support for the quantitative results, while the distinction between the two visualization procedures should be noted: t-SNE is applied only to the high-dimensional conventional representation, whereas the Node2Vec points are plotted directly in their learned two-dimensional embedding space.

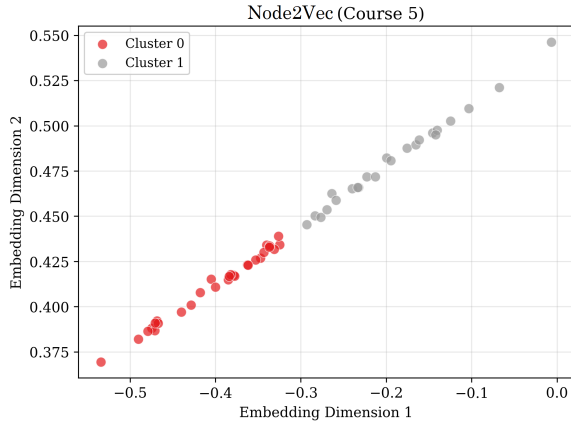
4.9 Runtime Analysis

Finally, we examine the computational cost of the complete Node2Vec pipeline. Table 9 reports the runtime of graph construction, random-walk generation, Word2Vec training, and the complete pipeline for each course. The complete pipeline requires less than nine seconds for all six courses. Word2Vec training is the dominant component of the runtime, accounting for approximately 80% of the total execution time, whereas graph construction and walk generation require comparatively little time.

For a dataset with n students and m courses, construction of the student co-enrollment graph from the enrollment matrix requires comparing all student pairs over all courses, yielding a cost of $O(n^2m)$. Random-walk generation requires



(a) t-SNE of the *Student-Course Rep.*



(b) Node2Vec embeddings ($d = 2$).

Fig. 9: Visualization of student representations for Course 5. (a) The conventional Student-Course representation is defined in a 32-dimensional space and is projected to two dimensions using t-SNE solely for visualization. (b) Node2Vec directly produces a two-dimensional representation, so no additional dimensionality-reduction method is required. In both panels, students are displayed according to their resulting section assignments.

$O(n\gamma t)$ operations, where γ is the number of walks per node and t is the walk length. The cost of Skip-Gram training is proportional to the total number of walk-context pairs and the embedding dimension, approximately $O(n\gamma td)$, where d is the embedding dimension. For the relatively small to moderate student populations considered in this study, the overall computational cost remains modest.

Table 9: Runtime analysis in seconds for the complete Node2Vec pipeline using $d = 2$, $t = 10$, $\gamma = 80$, $w = 5$, and $\epsilon = 30$.

Course	Students	Graph	Walks	Word2Vec	Total
1	74	0.024	0.263	8.067	8.355
2	51	0.011	0.168	5.471	5.650
3	48	0.009	0.144	4.584	4.738
4	49	0.070	0.145	4.785	5.000
5	52	0.009	0.153	4.830	4.992
6	67	0.016	0.218	6.564	6.798

Overall, the experiments demonstrate that the Node2Vec representation provides substantially improved clustering quality relative to the conventional Student-Course representation in the evaluated setting. The improvement is observed across multiple clustering algorithms and evaluation metrics, while the stability analysis indicates that the resulting clusterings are highly reproducible. The sensitivity experiments further show that effective representations can be obtained in a very low-dimensional space. The runtime analysis indicates that the complete pipeline can be executed within a few seconds for the course sizes considered in this study.

5 Discussion

The experimental results provide consistent evidence that graph representation learning substantially improves the quality of student sectioning. Across all evaluated courses and clustering algorithms, Node2Vec produces markedly higher clustering quality than both the conventional student-course representation and spectral clustering.

A key explanation for this improvement lies in how relational information is captured. The conventional representation describes students only through direct course enrollments, representing relationships implicitly. In contrast, the co-enrollment graph explicitly models students as nodes connected by weighted edges. Random walks on this graph expose the embedding model to sequences of structurally related students,

allowing Node2Vec to encode higher-order proximity and contextual relationships not expressed in the original enrollment matrix.

The baseline experiments provide an important control. While PCA substantially improves clustering over the conventional representation by reducing dimensionality, Node2Vec achieves significantly higher scores even when projected to the same two-dimensional space. This indicates that the performance gain is not merely a byproduct of dimensionality reduction, but rather stems from the structural graph-context captured by the random-walk-based learning. Similarly, the poor performance of direct spectral clustering confirms that simply applying a graph partitioning algorithm to the co-enrollment graph is insufficient; learning a task-oriented representation is critical.

An important finding is that the best performance is achieved with very low-dimensional embeddings, particularly $d = 1$ and $d = 2$. This suggests that the structural patterns relevant to student sectioning in this dataset are highly compressible. While we reported the main baseline comparisons at $d = 2$ to facilitate 2D visualization and matched comparison with PCA, the global optimum at $d = 1$ further highlights the efficiency of the learned representations.

The improvement is largely independent of the clustering algorithm, demonstrating that Node2Vec serves as an effective preprocessing stage compatible with standard institutional clustering pipelines. Furthermore, the high stability across random seeds and the minimal computational cost (under 9 seconds per course) demonstrate the framework’s readiness for real-world educational scheduling applications.

6 Conclusion

This paper revisited the student sectioning problem from a graph representation learning perspective. By constructing a weighted student co-enrollment graph and employing Node2Vec, we learned low-dimensional student embeddings that significantly improve clustering quality compared to conventional representations and spectral clustering.

Extensive experiments on six real-world university courses demonstrated that Node2Vec consistently outperforms the strongest baselines, including PCA followed by KMeans, across all

evaluated courses with large effect sizes. The proposed framework proves that the structural information captured by biased random walks provides additional value beyond generic dimensionality reduction. Furthermore, the method is highly robust, showing minimal sensitivity to hyperparameter tuning, high stability across random seeds, and very low computational cost.

The framework can be seamlessly incorporated into existing timetabling pipelines without modifying downstream clustering algorithms. Future work will explore the application of this framework to larger, multi-institutional datasets, sparser co-enrollment graphs, and the integration of graph neural networks to incorporate additional student attributes. Ultimately, integrating graph-based sectioning with constraint programming could allow direct evaluation of its impact on complete timetable quality.

7 Declarations

Declaration of Generative AI and AI-assisted Technologies in the Writing Process

During the preparation of this work, the author used free AI-assisted tools and services, including Sider, Mimo and Buffy (via Freebuff), OpenAI ChatGPT, and Z.AI, for various aspects of the research and manuscript preparation process, including writing and editing assistance, code development, manuscript organization, and scientific presentation. All generated content and outputs were reviewed, verified, and edited by the author as appropriate. The author takes full responsibility for the accuracy, integrity, and final content of the publication.

Competing interests

The author has no relevant financial or non-financial interests to disclose.

Code and Data availability

The Python code and data used in this work are available at <https://github.com/mamintoosi/Node2Vec-SSP>.

A Node2Vec Random Walk Generation

Algorithm 2 details the generation of biased random walks used in this study.

References

- [1] Amintoosi, M., Yazdi, H.S., Haddadnia, J.: Fuzzy student sectioning. In: PATAT04: Practice and Theory of Automated Timetabling, USA, pp. 421–424 (2004)
- [2] Amintoosi, M., Haddadnia, J.: Feature selection in a fuzzy student sectioning algorithm. In: Burke, E., Trick, M. (eds.) Practice and Theory of Automated Timetabling V, pp. 147–160. Springer, Berlin, Heidelberg (2005). Lecture Notes in Computer Science (LNTCS, volume 3616)
- [3] Amintoosi, M., Fathy, M., Mozayani, N., Rahmani, A.T.: A fish school clustering algorithm: Applied to student sectioning problem. Dynamics of Continuous Discrete & Impulse Systems, series B: Applications and Algorithms **2**, 696–699 (2007). Post Proceeding of LSMS2007, Life System Modeling and Simulation 2007, China
- [4] Aubin, J., Ferland, J.A.: A large scale timetabling problem. Computers & Operations Research **16**(1), 67–77 (1989)
- [5] Hertz, A., Robert, V.: Constructing a course schedule by solving a series of assignment type problems. European Journal of Operational Research **108**(3), 585–603 (1998)
- [6] Carter, M.W.: A comprehensive course timetabling and student scheduling system at the university of waterloo. In: Burke, E., Erben, W. (eds.) Practice and Theory of Automated Timetabling III, pp. 64–82. Springer, Berlin, Heidelberg (2001)
- [7] Alvarez-Valdes, R., Crespo, E., Tamarit, J.M.: Assigning students to course sections using tabu search. Annals of Operations Research **96**(1), 1–16 (2000)
- [8] Müller, T., Murray, K.: Comprehensive approach to student sectioning. Annals of Operations Research **181**(1), 249–269 (2010) <https://doi.org/10.1007/s10479-010-0735-9>
- [9] Perozzi, B., Al-Rfou, R., Skiena, S.: Deepwalk: Online learning of social representations. In: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 701–710 (2014)
- [10] Grover, A., Leskovec, J.: node2vec: Scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 855–864 (2016). <https://doi.org/10.1145/2939672.2939754>
- [11] Feldman, R., Golumbic, M.C.: Optimization algorithms for student scheduling via constraint satisfaction. The Computer Journal **33**(4), 356–364 (1990)
- [12] Qi, X.: Weighted jump in random walk graph sampling. Neurocomputing **586**, 127581 (2024)
- [13] Al Jreidy, M., *et al.*: Clustering using graph convolution networks. Progress in Artificial Intelligence (2024) <https://doi.org/10.1007/s13748-023-00310-z>
- [14] Solgi, M., *et al.*: Improving graph prototypical network using active learning. Progress in Artificial Intelligence (2022) <https://doi.org/10.1007/s13748-022-00293-3>
- [15] Luo, Z., *et al.*: Improving link prediction in social networks using local and global structure. Progress in Artificial Intelligence **11**(1), 79–92 (2022) <https://doi.org/10.1007/s13748-021-00261-3>
- [16] Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space. In: 1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings (2013)
- [17] Zaki, M.J., Meira, W., Meira, W.: Data

Algorithm 2 GenerateRandomWalks (Node2Vec)

Require: Graph $G = (V, E, W)$, walk length t , number of walks per node γ , return parameter p , in-out parameter q

Ensure: A list $\mathcal{W}$ containing all sequences of random walks

```
1:  $\mathcal{W} \leftarrow []$  ▷ Initialize list to store walks
2: for each graph node as  $start$  do
3:   for  $k = 1$  to  $\gamma$  do ▷ Number of walks per node
4:      $walk \leftarrow [start]$  ▷ Start the walk at the current node
5:      $prev \leftarrow \text{null}, cur \leftarrow start$ 
6:     for  $j = 1$  to  $t$  do ▷ Walk length
7:        $N(cur) \leftarrow \text{neighbors of } cur \text{ in } G$ 
8:       if  $prev = \text{null}$  then ▷ First step: uniform
9:          $next \leftarrow \text{uniform random choice from } N(cur)$ 
10:      else
11:        ▷ Compute biased transition probabilities
12:        for each neighbor  $z \in N(cur)$  do
13:           $d \leftarrow \text{shortest-path distance from } z \text{ to } prev \text{ in } G$ 
14:           $\alpha_{pq} \leftarrow \begin{cases} 1/p, & d = 0 \\ 1, & d = 1 \\ 1/q, & d = 2 \end{cases}$ 
15:           $\pi(z) \leftarrow \alpha_{pq} \cdot w_{cur,z}$  ▷ Unnormalized weight
16:        end for
17:        Normalize  $\pi$  into a probability distribution over  $N(cur)$ 
18:         $next \leftarrow \text{sample from } N(cur) \text{ according to } \pi$ 
19:      end if
20:       $walk \leftarrow walk + [next]$  ▷ Append next node to walk
21:       $prev \leftarrow cur, cur \leftarrow next$ 
22:    end for
23:     $\mathcal{W} \leftarrow \mathcal{W} + [walk]$  ▷ Store the walk
24:  end for
25: end for
26: return  $\mathcal{W}$ 
```

Mining and Machine Learning: Fundamental Concepts and Algorithms. Cambridge University Press, University of Cambridge (2020)